

Technical Specifications Document

Internship & Job Listing Portal

Project type	Internship mini project (full stack, beginner level)
Duration	2 weeks
Document version	1.0
Based on	PRD v2: Internship & Job Listing Portal
Stack	React, Node.js + Express, MongoDB

1. Introduction

This document explains how the Internship & Job Listing Portal will be built. The PRD says *what* the app does. This document says *how*: the tools, the database, the APIs, the pages and the rules behind them.

The goal is to keep things simple enough to finish in two weeks, while still showing a proper frontend, backend, database and API integration, since the project is judged on all of them together.

1.1 Scope

- Applicants browse, search (by title), filter (by domain), and open opportunities.
- Applicants apply through an in-app form and get a unique application ID they can use later to reopen the application.
- One admin logs in, manages opportunities (add, edit, delete) and views all applications.
- Out of scope for now: user login, resume file upload, email notifications.

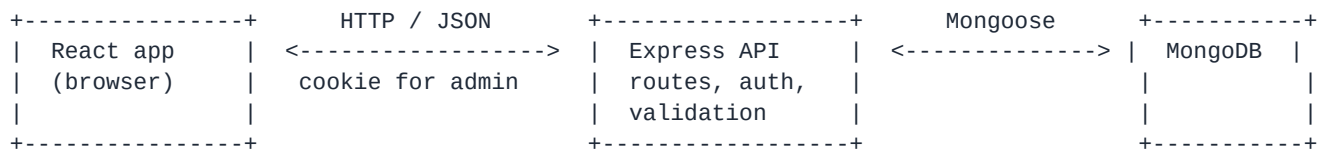
2. Tech Stack

Layer	Choice	Why
Frontend	React (Vite), React Router, Axios	Matches the assignment suggestion. Vite is quick to set up.
Backend	Node.js, Express.js	REST APIs, as suggested in the assignment.
Database	MongoDB with Mongoose	Document model fits opportunities and applications well. Mongoose gives schemas and validation.
Auth	express-session + connect-mongo, bcrypt	Session cookie login for the admin. Passwords are stored hashed.
Config	dotenv, cors	Secrets in .env. CORS lets the React app talk to the API with cookies.
Tools	Git/GitHub, Postman, MongoDB Compass	Version control, API testing, viewing data.

Styling is not fixed by the PRD. Plain CSS or CSS modules is enough; pick one and stay consistent.

3. System Architecture

A standard three-tier setup. The browser never touches the database directly.



- The React app calls the API using Axios (with `withCredentials: true` so the session cookie is sent).
- Express handles routing, input checks, admin auth and database access.
- MongoDB stores three collections: admins, opportunities, applications (plus a sessions collection created by connect-mongo).

3.1 Folder structure

```
job-portal/
  client/                                (React app)
    src/
      pages/                            Home, OpportunityDetails, ApplySuccess, ViewApplication,
                                          AdminLogin, AdminOpportunities, AdminOpportunityForm,
                                          AdminApplications
      components/                        Navbar, OpportunityCard, SearchBar, DomainFilter, ProtectedRoute
      api/                              axios instance and API helper functions
      App.jsx, main.jsx
    server/
      src/
        models/                         Admin.js, Opportunity.js, Application.js
        routes/                         opportunities.js, applications.js, admin.js
        middleware/                     requireAdmin.js, errorHandler.js
        utils/                          generateApplicationId.js
        config/                         db.js
        seed/                           seedAdmin.js
      app.js, server.js
    .env.example
    README.md
```

4. Database Design

Database: MongoDB. Three main collections.

4.1 admins

Field	Type	Rules
_id	ObjectId	Auto
email	String	Required, unique, stored lowercase
passwordHash	String	Required. bcrypt hash, never the plain password
createdAt	Date	Auto (timestamps)

4.2 opportunities

Field	Type	Rules
_id	ObjectId	Auto

Field	Type	Rules
title	String	Required. Used by the search
companyName	String	Required
type	String	Required. Enum: Job, Internship
domain	String	Required. Enum: Web Development, Data & AI, UI/UX Design, Marketing, Other
location	String	Required
experience	String	Required. Free text, e.g. "Fresher" or "1-2 years"
description	String	Required
applicationLink	String	Required. A URL shown on the details page
createdAt / updatedAt	Date	Auto. List is sorted by createdAt, newest first

Index: { title: 1 } is optional here. With a small data set, a case-insensitive regex search is fine.

4.3 applications

Field	Type	Rules
_id	ObjectId	Auto
applicationId	String	Required, unique. Format like APP-7K3F9Q, generated by the server
opportunity	ObjectId (ref)	Required. Points to opportunities._id
name	String	Required
phone	String	Required. Stored as text
email	String	Required. Trimmed and lowercased before saving
resumeLink	String	Optional. A URL, not a file
message	String	Optional
createdAt	Date	Auto

Unique compound index: { opportunity: 1, email: 1 }, unique: true. This is what blocks a second application from the same email for the same opportunity. The database enforces it, so it still works if two requests arrive at the same moment.

4.4 Relationships

admins (standalone)
opportunities 1 ----- * applications (application.opportunity -> opportunity._id)

5. API Specification

Base path: /api. All requests and responses use JSON. Routes marked **Admin** need a valid session and return 401 without one.

5.1 Public endpoints

Method	Endpoint	Purpose	Notes
GET	/api/opportunities	List opportunities	Query: <i>search</i> (title only), <i>domain</i> . Newest first, no pagination.

Method	Endpoint	Purpose	Notes
GET	/api/opportunities/:id	One opportunity	404 if not found.
POST	/api/applications	Submit an application	Body: opportunityId, name, phone, email, resumeLink?, message?. Returns 201 with applicationId. 409 if duplicate.
GET	/api/applications/:applicationId	Reopen a saved application	Looked up by the short code, not by _id. Includes opportunity title and company.

5.2 Admin endpoints

Method	Endpoint	Purpose	Notes
POST	/api/admin/login	Log in	Body: email, password. Sets session cookie. 401 on wrong details.
POST	/api/admin/logout	Log out	Destroys the session.
GET	/api/admin/me	Check login state	Used by the React app on page load. 401 if not logged in.
POST	/api/opportunities	Add opportunity	Admin. Returns 201.
PUT	/api/opportunities/:id	Edit opportunity	Admin. Returns updated record.
DELETE	/api/opportunities/:id	Delete opportunity	Admin. See section 8 for what happens to its applications.
GET	/api/admin/applications	All applications	Admin. One list, newest first, each with its opportunity title.

5.3 Response and error format

Success returns the data directly. Errors always use the same shape so the frontend can show them easily:

```
{ "message": "You have already applied to this opportunity." }
```

```
400 required field missing / bad request
401 not logged in, or wrong login details
404 opportunity or application not found
409 duplicate application
500 unexpected server error
```

6. Authentication (Admin)

- **Login fields:** email and password.
- **Method:** session cookie. After a correct login, the server stores the admin's id in the session and sends a cookie to the browser.
- **Session store:** MongoDB, via connect-mongo, so sessions survive a server restart.
- **Cookie settings:** httpOnly on, sameSite 'lax', secure on only in production, expires after a set time (for example 1 day).
- **Password:** hashed with bcrypt. Login compares with bcrypt.compare. Wrong email and wrong password give the same message.
- **Protection:** a requireAdmin middleware checks the session and blocks the admin routes. On the frontend, a ProtectedRoute component redirects to the login page if /api/admin/me fails.

6.1 Creating the first admin

There is no sign-up page. The first admin is created once by a seed script:

```
cd server
node src/seed/seedAdmin.js      (reads ADMIN_EMAIL and ADMIN_PASSWORD from .env)
```

The script hashes the password, creates the admin if none exists with that email, and exits. Running it twice does not create a duplicate.

7. Frontend Design

7.1 Pages and routes

Route	Page	Who	What it does
/	Home	Everyone	Opportunity list, title search box, domain dropdown.
/opportunities/:id	Opportunity details	Everyone	Full details, Application Link, in-app apply form.
/application/:applicationId	View application	Everyone	Shows a saved application. Also the page the confirmation points to.
/admin/login	Admin login	Admin	Email and password form.
/admin/opportunities	Manage opportunities	Admin	Table with edit and delete, plus an Add button.
/admin/opportunities/new and /:id/edit	Opportunity form	Admin	Add or edit using one shared form.
/admin/applications	Applications	Admin	One list of all applications and which opportunity each is for.

7.2 Behaviour details

- **Search and filter:** both are sent to the same endpoint (?search=react&domain=web Development). Debounce the search input a little so it does not call the API on every key press.
- **After applying:** show the application ID clearly with a copy button and a link to the view page. Tell the applicant to save it.
- **States to handle on every page:** loading, empty list ("No opportunities found"), and API errors.
- **Forms:** required fields are marked. Only a filled/not-filled check is applied (see section 8).

8. Business Rules and Validation

Rule	How it is handled
Application required fields	name, phone, email must be non-empty (checked on the frontend and again on the server). Nothing else is format-checked, as decided.
Optional fields	resumeLink and message can be empty.
Duplicate application	Same email and same opportunity is rejected with 409. Email is trimmed and lowercased first so "A@x.com" and "a@x.com " count as the same.
Application ID	Prefix "APP-" plus 6 characters from a set without look-alike characters (no 0/O, 1/I). Generated with Node's crypto module. If the random code already exists, generate another.

Rule	How it is handled
Search	Title only, case-insensitive. Special characters in the search text are escaped before building the regex.
Domain and Type	Only the allowed values are accepted by the Mongoose enum.
Deleting an opportunity	Suggested default: the opportunity is deleted but its applications are kept. The admin list shows "Opportunity removed" for them. The PRD does not say what should happen, so confirm with the mentor.

9. Security Basics

- Passwords hashed with bcrypt, never logged or returned by any API.
- Secrets (session secret, DB URL, admin seed details) live in .env, and .env is in .gitignore. Only .env.example is committed.
- CORS limited to the frontend's address, with credentials allowed.
- Mongoose schemas and the enum checks stop unexpected fields from being saved. Only pick known fields from req.body.
- Search text is escaped to avoid regex abuse.
- The public application lookup uses the short random ID. It is hard to guess but not secret-grade. That is acceptable here because the PRD asks for reopening by ID/link without login.

10. Environment Variables

```
# server/.env.example
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/job_portal
SESSION_SECRET=change_this_to_a_long_random_string
CLIENT_ORIGIN=http://localhost:5173
ADMIN_EMAIL=admin@example.com
ADMIN_PASSWORD=choose_a_strong_password
```

```
# client/.env
VITE_API_URL=http://localhost:5000/api
```

11. Testing Plan

Manual testing is enough for this project. Use Postman for the API first, then test through the UI.

Area	What to check
Opportunities	List loads newest first. Search by title works. Domain filter works. Both together work. Details page opens. Bad id returns 404.
Applications	Missing name, phone or email is rejected. Valid application returns an ID. Same email + same opportunity returns 409. Same email on a different opportunity is allowed. The ID opens the saved application.
Admin auth	Wrong password fails. Correct login sets a cookie. Admin routes return 401 when logged out. Logout really ends access. Session survives a page refresh.
Admin CRUD	Add, edit and delete all work and show up on the public list straight away.
UI	Loading, empty and error states show properly. Layout works on a phone-sized screen.

12. Development Plan

Follows the assignment timeline, with admin login placed in the weeks as the PRD asks.

Week	Backend	Frontend
Week 1	Project setup, MongoDB connection, the three models, opportunity APIs (list, details, search, filter), seed script, admin login and session.	React setup, folder structure, router, home page with list, details page, search and filter UI (using sample data until the API is ready).
Week 2	Application API with duplicate check and ID generation, admin CRUD and applications API, error handling.	Connect everything to the API, apply form and confirmation page, view-application page, admin pages, UI cleanup.
End of week 2	Test everything, fix bugs.	README.md, project report, screenshots, final presentation.

13. Deployment

TBD. The PRD leaves deployment undecided. The code is written so that moving online later only needs environment variable changes (database URL, client origin, API URL, secure cookies). Revisit this once the mentor confirms whether it is expected.

14. Open Items

#	Item	Current plan
1	Deployment expected, and where?	TBD (section 13)
2	What happens to applications when an opportunity is deleted?	Keep them (section 8). Confirm with mentor.
3	Format and length of the project report and final presentation	Not decided yet
4	Should admin sessions have a specific timeout?	1 day, unless told otherwise

15. Final Deliverables Checklist

- GitHub repository and complete source code
- Database structure (section 4 can be reused in the report)
- README.md with setup steps, env variables and the seed command
- Project report, screenshots, live demo (if deployed), final presentation